МІНІСТЕРСТВО НАУКИ ТА ОСВІТИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

ЗВІТ

до лабораторної роботи № 1

з дисципліни «Основи інформаційних систем та технологій»

Розробка елементів експертної системи продукційного типу,
яка реалізує пряме виведення

Виконав(-ла):
студент гр. КН-726
АНІСОВ Олександр
Перевірив(-ла):
ст. викл. каф. ІСТ
КОНДРАТОВ О.М.

Харків
2026

ЗМІСТ

Вступ 3

Мета роботи та постановка завдання 5

1 Короткі теоретичні відомості 6

2 Опис предметної області 7

3 База знань продукційного типу 8

4 Структури даних і алгоритм прямого виведення 10

5 Програмна реалізація мовою Python 11

6 Перевірка роботи експертної системи 15

7 Аналіз результатів 17

Висновки 18

Список джерел інформації 19

Додаток А Альтернативний титульний аркуш 20

Додаток Б Програмний код 22

Додаток В Математичний опис логічного правила 30

ВСТУП

У сучасних умовах розвитку інформаційних технологій важливе значення має автоматизація процесів аналізу даних, прийняття рішень та діагностики технічних систем. Одним із засобів розв’язання таких завдань є експертні системи, які дають змогу формалізувати знання предметної області та використовувати їх для отримання логічних висновків на основі відомих фактів.

Експертні системи продукційного типу є одним із підходів до подання знань, за якого інформація описується у формі правил виду «ЯКЩО – ТО». Такі правила визначають умови, за яких наявні факти можуть бути використані для формування нових висновків. Важливою складовою таких систем є механізм логічного виведення, який забезпечує послідовне застосування правил відповідно до поточного стану робочої пам’яті.

Одним із поширених способів логічного виведення є пряме виведення, яке передбачає початок процесу з відомих фактів та послідовне отримання нових фактів шляхом застосування відповідних правил. Такий підхід дає змогу реалізувати автоматизований аналіз ситуацій, у яких необхідно визначити можливі причини проблем на основі спостережуваних ознак.

У цій лабораторній роботі предметною областю обрано первинну діагностику типових несправностей персонального комп’ютера. Аналізуються такі ознаки, як наявність живлення, робота вентиляторів, наявність сигналу на дисплеї, завантаження операційної системи, виявлення накопичувача та доступність мережі. На основі цих даних формується база продукційних правил, за допомогою якої система визначає можливі проблеми.

Для реалізації експертної системи використовується мова програмування Python. Програмна реалізація передбачає створення структури для зберігання продукційних правил, організацію робочої пам’яті, використання черги для

послідовного опрацювання фактів і формування протоколу логічних висновків. Це дає змогу не лише отримувати результат діагностики, а й простежувати послідовність застосування правил, що привели до відповідного висновку.

Метою лабораторної роботи є вивчення принципів побудови експертних систем продукційного типу та розроблення програмної реалізації механізму прямого логічного виведення для первинної діагностики типових несправностей персонального комп’ютера.

Для досягнення поставленої мети необхідно виконати такі завдання дослідити структуру та основні компоненти експертної системи продукційного типу, сформувати базу знань у вигляді продукційних правил для обраної предметної області,розробити алгоритм прямого логічного виведення, реалізувати механізм виведення мовою Python із використанням робочої пам’яті та черги фактів, забезпечити формування діагностичних висновків і відображення ланцюга спрацювання правил, перевірити коректність роботи системи на тестових наборах вихідних фактів та проаналізувати отримані результати.

МЕТА РОБОТИ ТА ПОСТАНОВКА ЗАВДАННЯ

Вивчити склад і принципи роботи експертної системи продукційного типу та реалізувати механізм прямого логічного виведення. Побудувати базу знань у формі продукційних правил, організувати робочу пам’ять, чергу змінних і протокол спрацювання правил, а також перевірити роботу системи на тестових прикладах.

Постановка завдання. Необхідно розробити експертну систему продукційного типу для обраної предметної області. База знань повинна містити від 6 до 10 правил, що відображають причинно-наслідкові зв’язки. Система має реалізовувати пряме виведення, підтримувати введення вихідних фактів, формувати остаточний результат і за вимогою показувати ланцюг логічних висновків.

1 КОРОТКІ ТЕОРЕТИЧНІ ВІДОМОСТІ

Експертна система складається з бази знань, робочої пам’яті та механізму логічного виведення. База знань містить продукційні правила виду «ЯКЩО умова, ТО висновок». Робоча пам’ять зберігає факти, відомі системі в поточний момент, а механізм виведення визначає, які правила можуть бути активовані [1-2].

За прямого виведення процес починається з відомих фактів. Після спрацювання правила новий факт додається до робочої пам’яті, після чого система перевіряє наступні правила. Для пошуку завширшки нові факти опрацьовуються в порядку їх надходження за принципом FIFO [3-5]. На рис. 1.1 використання гіта та клонування репозиторію.

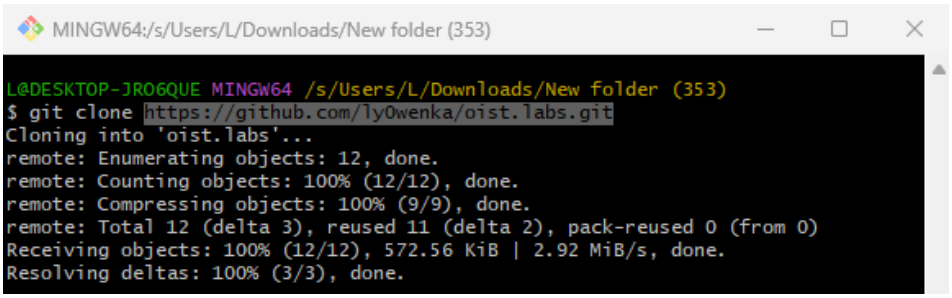


Рисунок 1.1 – Використання гіта

На рис. 1.2 узагальнена структура експертної системи.



Рисунок 1.2 – Узагальнена структура експертної системи

2 ОПИС ПРЕДМЕТНОЇ ОБЛАСТІ

Предметною областю обрано первинну діагностику несправностей персонального комп’ютера. Користувач повідомляє спостережувані ознаки: чи вмикається живлення, чи обертаються вентилятори, чи є зображення, чи завантажується операційна система, чи визначається накопичувач і чи доступна мережа. На основі цих фактів експертна система формує один або декілька діагностичних висновків.

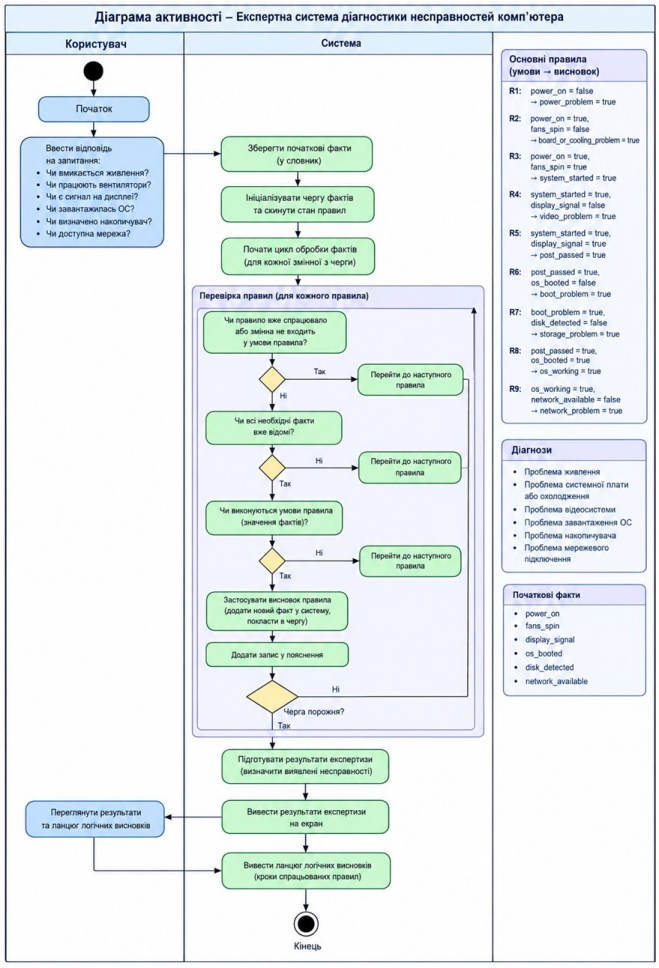


Рисунок 2.1 – Фрагмент дерева логічних висновків для діагностики ПК

3 БАЗА ЗНАНЬ ПРОДУКЦІЙНОГО ТИПУ

Для експертної системи сформовано дев’ять продукційних правил в таблиці 3.1.

Таблиця 3.1 – База знань експертної системи

№	Умовна частина	Висновок
R1	живлення = ні	проблема_живлення = так.
R2	живлення = так І вентилятори = ні	проблема_плати_або_охолодження = так.
R3	живлення = так І вентилятори = так	система_стартує = так.
R4	система_стартує = так І сигнал_дисплея = ні	проблема_відеосистеми = так.

Кінець таблиці 3.1

№	Умовна частина	Висновок
R5	система_стартує = так І сигнал_дисплея = так	post_пройдено = так.
R6	post_пройдено = так І ос_завантажилась = ні	проблема_завантаження = так.
R7	проблема_завантаження = так І диск_визначено = ні	проблема_накопичувача = так.
R8	post_пройдено = так І ос_завантажилась = так	ос_працює = так.
R9	ос_працює = так І мережа_доступна = ні	мережева_проблема = так.

4 СТРУКТУРИ ДАНИХ І АЛГОРИТМ ПРЯМОГО ВИВЕДЕННЯ

У програмі база знань подається списком об’єктів Rule. Кожне правило містить набір умов, висновок та ознаку fired, яка показує, чи було правило вже активовано. Робоча пам’ять реалізована словником facts, де ключем є ім’я змінної, а значенням її поточне значення. Черга queue зберігає нові факти в порядку FIFO, а список explanation накопичує текстовий протокол спрацювання правил.

1 Записати вихідні факти користувача до робочої пам’яті та черги.
2 Взяти з початку черги черговий факт.
3 Переглянути правила, в умовах яких зустрічається ця змінна.
4 Для кожного правила перевірити, чи відомі всі його умови та чи всі вони істинні.
5 Якщо правило істинне і ще не спрацьовувало, додати його висновок до робочої пам’яті й у кінець черги.
6 Записати спрацювання правила до ланцюга пояснень.
7 Повторювати кроки 2–6, поки черга не стане порожньою.
8 Вивести діагностичні висновки та ланцюг логічного виведення.

5 ПРОГРАМНА РЕАЛІЗАЦІЯ МОВОЮ PYTHON

Програмна реалізація

```
from dataclasses import dataclass
from collections import deque

@dataclass
class Rule:
    name: str
    conditions: dict
    conclusion: tuple
    fired: bool = False

RULES = [
    Rule(
        "R1",
        {"power_on": False},
        ("power_problem", True)
    ),

    Rule(
        "R2",
        {"power_on": True, "fans_spin": False},
        ("board_or_cooling_problem", True)
    ),

    Rule(
        "R3",
        {"power_on": True, "fans_spin": True},
        ("system_started", True)
    ),

    Rule(
        "R4",
        {"system_started": True, "display_signal": False},
        ("video_problem", True)
    ),

    Rule(
        "R5",
        {"system_started": True, "display_signal": True},
        ("post_passed", True)
    ),

    Rule(
        "R6",
        {"post_passed": True, "os_booted": False},
        ("boot_problem", True)
    ),

    Rule(
        "R7",
```

```
        {"boot_problem": True, "disk_detected": False},
        ("storage_problem", True)
    ),

    Rule(
        "R8",
        {"post_passed": True, "os_booted": True},
        ("os_working", True)
    ),

    Rule(
        "R9",
        {"os_working": True, "network_available": False},
        ("network_problem", True)
    ),
]

def infer(initial_facts):
    facts = dict(initial_facts)

    # Черга фактів, які потрібно обробити
    queue = deque(initial_facts.keys())

    explanation = []

    # Скидання стану правил
    for rule in RULES:
        rule.fired = False

    while queue:
        variable = queue.popleft()

        for rule in RULES:

            # Якщо правило вже спрацювало
            # або поточна змінна не бере участі в правилі
            if rule.fired or variable not in rule.conditions:
                continue

            # Перевіряємо, чи всі необхідні факти вже відомі
            all_known = all(
                k in facts
                for k in rule.conditions
            )

            # Головне виправлення:
            # перевіряємо значення тільки якщо всі факти існують
            if all_known and all(
                facts[k] == v
                for k, v in rule.conditions.items()
            ):

                result_var, result_value = rule.conclusion

                rule.fired = True
```

```
                if result_var not in facts:
                    facts[result_var] = result_value
                    queue.append(result_var)

                explanation.append(
                    f"{rule.name}: {rule.conditions} -> "
                    f"{result_var}={result_value}"
                )

    return facts, explanation

def ask_yes_no(question):
    while True:
        value = input(
            question + " [так/ні]: "
        ).strip().lower()

        if value in ("так", "т", "yes", "y"):
            return True

        if value in ("ні", "н", "no", "n"):
            return False

        print("Введіть 'так' або 'ні'.")

def main():

    initial = {
        "power_on": ask_yes_no(
            "Чи вмикається живлення?"
        ),

        "fans_spin": ask_yes_no(
            "Чи працюють вентилятори?"
        ),

        "display_signal": ask_yes_no(
            "Чи є сигнал на дисплеї?"
        ),

        "os_booted": ask_yes_no(
            "Чи завантажилась ОС?"
        ),

        "disk_detected": ask_yes_no(
            "Чи визначено накопичувач?"
        ),

        "network_available": ask_yes_no(
            "Чи доступна мережа?"
        ),
    }

    facts, explanation = infer(initial)

    diagnoses = {
```

```
        "power_problem":
            "Проблема живлення",

        "board_or_cooling_problem":
            "Проблема системної плати або охолодження",

        "video_problem":
            "Проблема відеосистеми",

        "boot_problem":
            "Проблема завантаження ОС",

        "storage_problem":
            "Проблема накопичувача",

        "network_problem":
            "Проблема мережевого підключення",
    }

    print("\nРезультат експертизи:")

    found = False

    for key, label in diagnoses.items():

        if facts.get(key) is True:
            print("-", label)
            found = True

    if not found:
        print(
            "Критичних несправностей за заданими "
            "правилами не виявлено."
        )

    print("\nЛанцюг логічних висновків (A chain of logical inferences):(")

    for step in explanation:
        print(step)

if __name__ == "__main__":
    main()
```

6 ПЕРЕВІРКА РОБОТИ ЕКСПЕРТНОЇ СИСТЕМИ

Для перевірки реалізації використано три тестові набори вихідних фактів. У кожному випадку система повинна отримати діагноз за допомогою послідовного спрацювання продукційних правил.

Таблиця 6.1 – Результати тестування

Тест	Вихідні факти	Ланцюг правил	Отриманий висновок
1	Живлення = ні	R1	Проблема живлення
2	Живлення = так; вентилятори = так; сигнал дисплея = так; ОС = ні; диск = ні	R3 → R5 → R6 → R7	Проблема завантаження ОС; проблема накопичувача

Продовження таблиці 6.1

Тест	Вихідні факти	Ланцюг правил	Отриманий висновок
3	Живлення = так; вентилятори = так; сигнал дисплея = так; ОС = так; мережа = ні	R3 → R5 → R8 → R9	Проблема мережевого підключення

Приклад роботи для тесту 2:

```
Чи вмикається живлення? [так/ні]: так
Чи працюють вентилятори? [так/ні]: так
Чи є сигнал на дисплеї? [так/ні]: так
Чи завантажилась ОС? [так/ні]: ні
Чи визначено накопичувач? [так/ні]: ні
Чи доступна мережа? [так/ні]: так

Результат експертизи:
- Проблема завантаження ОС
- Проблема накопичувача

Ланцюг логічних висновків:
R3: power_on=True, fans_spin=True -> system_started=True
R5: system_started=True, display_signal=True -> post_passed=True
R6: post_passed=True, os_booted=False -> boot_problem=True
R7: boot_problem=True, disk_detected=False -> storage_problem=True
```

7 АНАЛІЗ РЕЗУЛЬТАТІВ

У всіх тестах отримані результати відповідають сформованій базі знань. У тесті 2 висновок утворюється не одним правилом, а послідовністю R3, R5, R6 і R7, що підтверджує роботу прямого виведення. Нові факти system_started, post_passed і boot_problem послідовно додаються до робочої пам’яті та черги, після чого стають підставою для активації наступних правил.

Реалізація відокремлює знання від механізму виведення: для розширення предметної області достатньо додати нові об’єкти Rule без зміни функції infer(). Це забезпечує модульність продукційної моделі та спрощує подальше доповнення системи.

ВИСНОВКИ

У лабораторній роботі розроблено експертну систему продукційного типу для первинної діагностики несправностей персонального комп’ютера. Сформовано базу знань з дев’яти правил, реалізовано робочу пам’ять, чергу FIFO та механізм прямого логічного виведення мовою Python. Система приймає вихідні факти, формує діагностичний висновок і відображає ланцюг правил, які привели до результату. Тестування показало коректне послідовне спрацювання правил і можливість одержання багатокрокових висновків.

СПИСОК ДЖЕРЕЛ ІНФОРМАЦІЇ

1 Giarratano J. C., Riley G. D. Expert Systems: Principles and Programming. 4th ed. Thomson Course Technology, 2005.

2 Russell S. J., Norvig P. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2021.

3 Rule-Based Inference (Forward and Backward Chaining). Living Books. URL: https://living-book.github.io/atlas/concepts/ai-guide-to-intelligent-systems/rule-based-inference/ (дата звернення: 13.10.2026).

4 CLIPS. CLIPS Documentation. URL: https://www.clipsrules.net/Documentation.html (дата звернення: 13.10.2026).

5 Python Software Foundation. collections — Container datatypes. URL: https://docs.python.org/uk/3/library/collections.html (дата звернення: 13.10.2026).

ДОДАТОК А

Альтернативний Титульний аркуш

Титульна сторінка це «обличчя» роботи, яке насамперед бачить викладач або екзаменаційна комісія. Неправильне оформлення може знизити враження навіть від якісного змісту. У цьому матеріалі розглянуто, як правильно оформити титульний аркуш дипломної роботи, яких помилок уникати та як використовувати практичний зразок. Ця інформація допоможе уникнути формальних помилок, заощадити час і вчасно підготувати дипломну роботу до захисту. Титульний аркуш дипломної роботи — перша сторінка академічної праці, що містить основну інформацію про студента, тему роботи, навчальний заклад, наукового керівника та інші необхідні відомості. Він підтверджує авторство та забезпечує оформлення документа відповідно до вимог закладу вищої освіти. Титульна сторінка є важливим елементом роботи, оскільки формує перше враження про студента як автора роботи. Правильно оформлений титульний аркуш свідчить про впорядкованість, відповідальність Основні вимоги до оформлення титульного аркуша зазвичай визначаються методичними рекомендаціями закладу вищої освіти. Водночас більшість українських університетів використовують подібний перелік обов’язкових елементів. На титульній сторінці зазначають повну офіційну назву закладу вищої освіти, назву факультету або інституту та кафедри, тему дипломної роботи, вид роботи й освітній рівень, ім’я та прізвище здобувача освіти, ім’я та прізвище наукового керівника із зазначенням його наукового ступеня та вченого звання за потреби, місто виконання роботи та рік її захисту. Для забезпечення належного вигляду титульного аркуша необхідно дотримуватися встановлених вимог до шрифту, розміру літер, вирівнювання тексту, міжрядкових інтервалів і розташування інформаційних блоків.

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

КАФЕДРА ІНФОРМАЦІЙНИХ СИСТЕМ ТА ТЕХНОЛОГІЙ

ЗВІТ

до лабораторної роботи № 1

з дисципліни «Основи інформаційних систем та технологій»

Розробка елементів експертної системи продукційного типу,
яка реалізує пряме виведення

Виконав(ла)

Прізвище Ім’я По-батькові

Перевірив(-ла)

посада Прізвище Ім’я По-батькові

Харків 2026

ДОДАТОК Б

Програмний код

```
from dataclasses import dataclass
from collections import deque

@dataclass
class Rule:
    name: str
    conditions: dict
    conclusion: tuple
    fired: bool = False

RULES = [
    Rule(
        "R1",
        {"power_on": False},
        ("power_problem", True)
    ),

    Rule(
        "R2",
        {"power_on": True, "fans_spin": False},
        ("board_or_cooling_problem", True)
    ),
```

```
Rule(
    "R3",
    {"power_on": True, "fans_spin": True},
    ("system_started", True)
),

Rule(
    "R4",
    {"system_started": True, "display_signal": False},
    ("video_problem", True)
),

Rule(
    "R5",
    {"system_started": True, "display_signal": True},
    ("post_passed", True)
),

Rule(
    "R6",
    {"post_passed": True, "os_booted": False},
    ("boot_problem", True)
),

Rule(
    "R7",
```

```
        {"boot_problem": True, "disk_detected": False},
        ("storage_problem", True)
    ),

    Rule(
        "R8",
        {"post_passed": True, "os_booted": True},
        ("os_working", True)
    ),

    Rule(
        "R9",
        {"os_working": True, "network_available": False},
        ("network_problem", True)
    ),
]

def infer(initial_facts):
    facts = dict(initial_facts)

    # Черга фактів, які потрібно обробити
    queue = deque(initial_facts.keys())

    explanation = []

    # Скидання стану правил
    for rule in RULES:
```

```
    rule.fired = False

while queue:
    variable = queue.popleft()

    for rule in RULES:

        # Якщо правило вже спрацювало
        # або поточна змінна не бере участі в правилі
        if rule.fired or variable not in rule.conditions:
            continue

        # Перевіряємо, чи всі необхідні факти вже відомі
        all_known = all(
            k in facts
            for k in rule.conditions
        )

        # Головне виправлення:
        # перевіряємо значення тільки якщо всі факти існують
        if all_known and all(
            facts[k] == v
            for k, v in rule.conditions.items()
        ):

            result_var, result_value = rule.conclusion
```

```
            rule.fired = True

            if result_var not in facts:
                facts[result_var] = result_value
                queue.append(result_var)

            explanation.append(
                f"{rule.name}: {rule.conditions} -> "
                f"{result_var}={result_value}"
            )

    return facts, explanation

def ask_yes_no(question):
    while True:
        value = input(
            question + " [так/ні]: "
        ).strip().lower()

        if value in ("так", "т", "yes", "y"):
            return True

        if value in ("ні", "н", "no", "n"):
            return False

        print("Введіть 'так' або 'ні'.")
```

```
def main():

    initial = {
        "power_on": ask_yes_no(
            "Чи вмикається живлення?"
        ),

        "fans_spin": ask_yes_no(
            "Чи працюють вентилятори?"
        ),

        "display_signal": ask_yes_no(
            "Чи є сигнал на дисплеї?"
        ),

        "os_booted": ask_yes_no(
            "Чи завантажилась ОС?"
        ),

        "disk_detected": ask_yes_no(
            "Чи визначено накопичувач?"
        ),

        "network_available": ask_yes_no(
            "Чи доступна мережа?"
        ),
    }
```

```
facts, explanation = infer(initial)

diagnoses = {
    "power_problem":
        "Проблема живлення",

    "board_or_cooling_problem":
        "Проблема системної плати або охолодження",

    "video_problem":
        "Проблема відеосистеми",

    "boot_problem":
        "Проблема завантаження ОС",

    "storage_problem":
        "Проблема накопичувача",

    "network_problem":
        "Проблема мережевого підключення",
}

print("\nРезультат експертизи:")

found = False
```

```
    for key, label in diagnoses.items():

        if facts.get(key) is True:
            print("-", label)
            found = True

    if not found:
        print(
            "Критичних несправностей за заданими "
            "правилами не виявлено."
        )

    print("\nЛанцюг логічних висновків (A chain of logical inferences):(")

    for step in explanation:
        print(step)

if __name__ == "__main__":
    main()
```

ДОДАТОК В

Математичний опис логічного правила

У розробленій експертній системі діагностики комп’ютера логічне виведення здійснюється на основі продукційних правил типу «ЯКЩО – ТО». Кожне правило перевіряє задані умови та, у разі їх виконання, формує відповідний висновок. Узагальнену математичну модель такого правила подано формулою:

$$A \wedge B \Rightarrow C, \tag{В.1}$$

де

(A, B) – умови або відомі факти;

$\wedge$ – логічне «І»;

C – результат логічного висновку;

$\Rightarrow$ – логічний перехід до висновку.